Estado del Arte de la Clasificación de Requerimientos No Funcionales con Desbalance de Clases: Aumentación de Datos, BERT y Modelos de Lenguaje de Gran Escala

Andres Lino Montes Mamani, Cristian Cabana Sulca, Alessandro Pastor Mamani Mamani
Escuela Profesional de Ingeniería de Sistemas
Universidad Peruana Unión, Juliaca, Perú
Asesor: Mg. Esteban Tocto Cano

***Resumen*—La clasificación automática de requerimientos no funcionales (NFR) sigue limitada por el desbalance extremo de clases. Este estado del arte analiza 145 estudios primarios publicados entre 2017 y 2026, el 62 % de ellos en 2025 o 2026. Los transformers (68 estudios) y los LLM (45) ya predominan sobre el aprendizaje automático clásico y el profundo. Solo 46 estudios tratan el desbalance o la escasez de datos. El remuestreo tipo SMOTE (10) y la generación con LLM (6) son las técnicas más usadas; EDA aparece en 2 estudios y TAOS en 1. En los estudios revisados no se identificó ninguno que compare TAOS, EDA y SMOTE-Tomek con el mismo clasificador BERT. Tampoco se identificó ninguno que contraste un BERT entrenado con datos aumentados frente a LLM usados como clasificadores. Además, el análisis de la partición pública de TAOS muestra que el 52 % de sus requerimientos de prueba está repetido en entrenamiento, y que sus 1,534 filas corresponden a 988 requerimientos distintos. Con esta evidencia se formulan cinco brechas y se describe el flujo experimental que las aborda.**

***Index Terms*—requerimientos no funcionales, desbalance de clases, aumentación de datos, BERT, modelos de lenguaje de gran escala, revisión de literatura**

I. Introducción

Los requerimientos no funcionales (NFR) describen atributos de calidad como la seguridad, el desempeño, la mantenibilidad o la portabilidad, y su identificación temprana en los documentos de especificación (SRS). Cleland-Huang et al. [1] propusieron uno de los primeros enfoques de clasificación automática de NFR y publicaron el conjunto PROMISE NFR [2], que sigue siendo la referencia del área. Desde entonces, la tarea se ha abordado con aprendizaje automático, aprendizaje profundo, transformers y, recientemente, modelos de lenguaje de gran escala (LLM).

El problema que persiste es el desbalance de clases. En la partición de entrenamiento que reportan Ben Ammar y Almatrafi [3], la clase funcional (F) tiene 557 muestras y portabilidad (PO) solo 2. Con tan pocos ejemplos, un clasificador favorece las clases frecuentes y no reconoce las minoritarias.

Este artículo revisa la literatura que fundamenta una investigación con tres objetivos: comparar tres métodos de aumentación de datos (TAOS, EDA y SMOTE-Tomek) entrenando BERT con cada uno; contrastar esos modelos con LLM usados como clasificadores; y ampliar después el conjunto con requerimientos de GitHub. La revisión responde cuatro preguntas:

- **PR1.** ¿Cómo ha evolucionado la clasificación automática de NFR y qué enfoques predominan?
- **PR2.** ¿Qué técnicas se usan contra el desbalance y la escasez de datos, y con qué clasificadores?
- **PR3.** ¿Cómo se usan los LLM en esta tarea y frente a qué se comparan?
- **PR4.** ¿Qué fuentes de datos se usan y cómo se amplían los conjuntos?

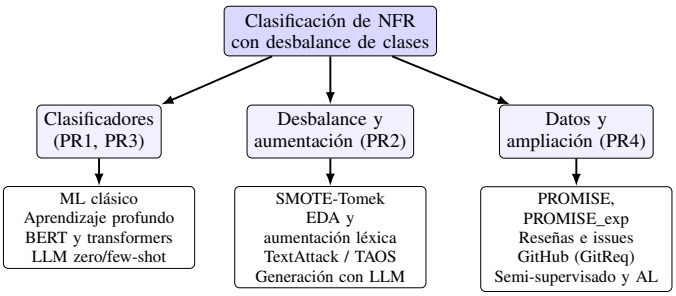


Fig. 1. Bloques temáticos de la revisión y preguntas que responde cada uno.

La Fig. 1 organiza los bloques temáticos de la revisión. La Sección II delimita el alcance, la III el panorama bibliométrico, la IV la evolución de los clasificadores (PR1 y PR3), la V el desbalance y la aumentación (PR2) y la VI la ampliación de conjuntos (PR4). Las Secciones VII a X presentan la comparación de estudios, la discusión de cada pregunta, las brechas y sus implicaciones para la investigación propuesta.

II. Alcance de la revisión

La revisión abarca los estudios primarios, publicados entre 2017 y 2026, que clasifican automáticamente requerimientos de software en tipos o en atributos de calidad mediante aprendizaje automático, aprendizaje profundo, transformers o LLM. Se analizaron 145 estudios, seleccionados en Scopus con criterios explícitos de inclusión y exclusión; el protocolo

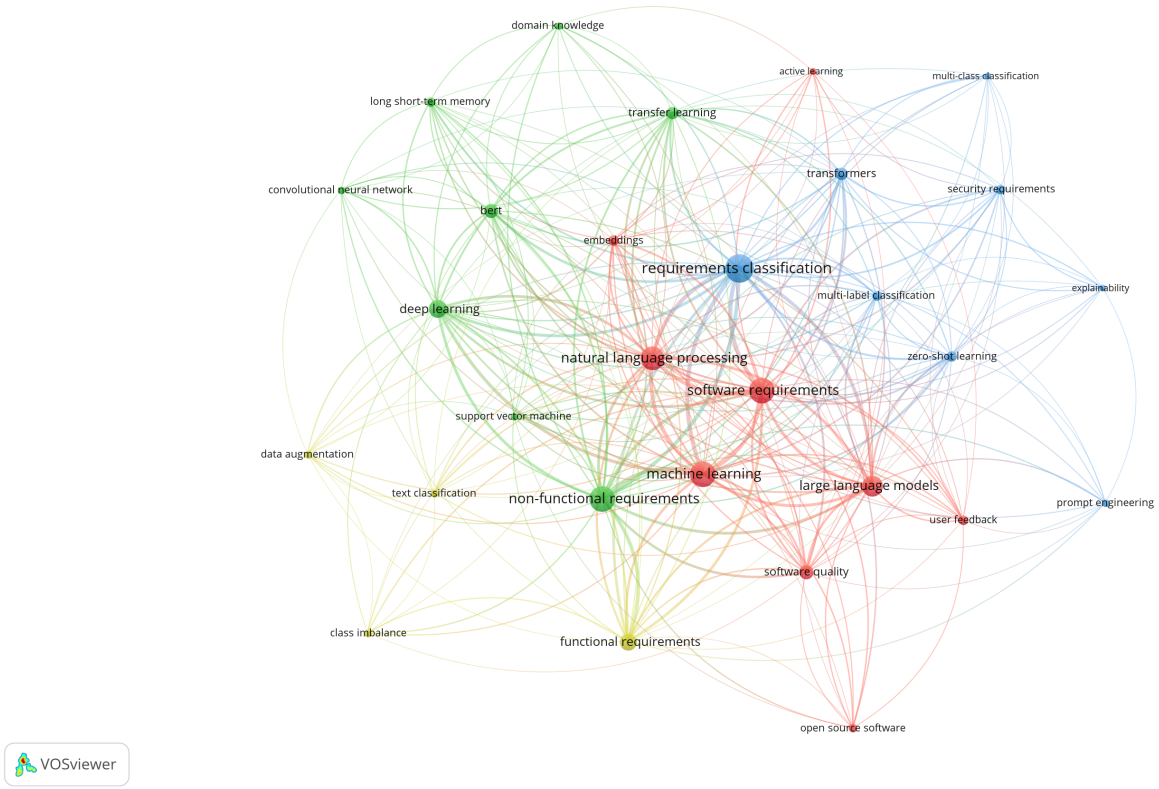


Fig. 2. Mapa de co-ocurrencia de palabras clave de los 145 estudios revisados, elaborado con VOSviewer [4]. Se muestran las 29 palabras clave con al menos cinco apariciones, tras unir sinónimos y excluir términos genéricos. El tamaño de cada nodo es proporcional al número de estudios que usan la palabra, las líneas indican co-ocurrencia y los colores identifican los cuatro clústeres.

completo de búsqueda y selección se presenta en la revisión sistemática asociada a esta investigación. Cada estudio se caracterizó según su enfoque principal, las técnicas que emplea contra el desbalance, la fuente del texto que clasifica y los conjuntos de datos que utiliza.

El análisis se organizó en torno a dos grupos de trabajos. El primero lo forman los trabajos base que más citan los 145 estudios. Los más citados del dominio son Kurtanović y Maalej [5] (71), NoRBERT [6] (65) y Cleland-Huang et al. [1] (48). El segundo lo forman tres estudios recientes, uno por cada eje temático, que sirvieron como artículos semilla (Tabla I).

Tabla I
ARTÍCULOS SEMILLA POR EJE TEMÁTICO

Eje	Artículo semilla	Fuente
Aumentación	TAOS [3]: aumentación con TextAttack y BERT sobre 12 clases	ETASR
LLM como clasificador	Binkhonain y Alfayez [7]: LLM con *prompts* frente a un transformer ajustado	Req. Eng.
BERT para NFR	Alhaizaey y Al-Mashari [8]: BERT, RoBERTa, XLNet y DistilBERT con *Focal Loss*	IEEE Access
Base co-citada	Kurtanović y Maalej [5]; NoRBERT [6]; Cleland-Huang et al. [1]	RE / Req. Eng.

III. PANORAMA BIBLIOMÉTRICO

III-A. Estructura temática

El mapa de co-ocurrencia de palabras clave (Fig. 2) agrupa el campo en cuatro clústeres:

- **Aprendizaje profundo y BERT** (verde): *non-functional requirements*, *deep learning*, BERT, *transfer learning*, LSTM, SVM y CNN. Reúne la línea de clasificadores supervisados sobre especificaciones.
- **Aprendizaje automático y LLM aplicados** (rojo): *machine learning*, *natural language processing*, *large language models*, *software quality*, *user feedback* y *open source software*. Concentra los estudios sobre texto de usuarios y proyectos abiertos.
- **Clasificación con transformers y *zero-shot*** (azul): *requirements classification*, *transformers*, *zero-shot learning*, *multi-label classification*, *security requirements*, *prompt engineering* y *explainability*.
- **Desbalance y aumentación** (amarillo): *functional requirements*, *class imbalance*, *text classification* y *data augmentation*. Es el clúster más pequeño y se ubica en la periferia del mapa.

La vista por año (Fig. 3) muestra el desplazamiento del campo.

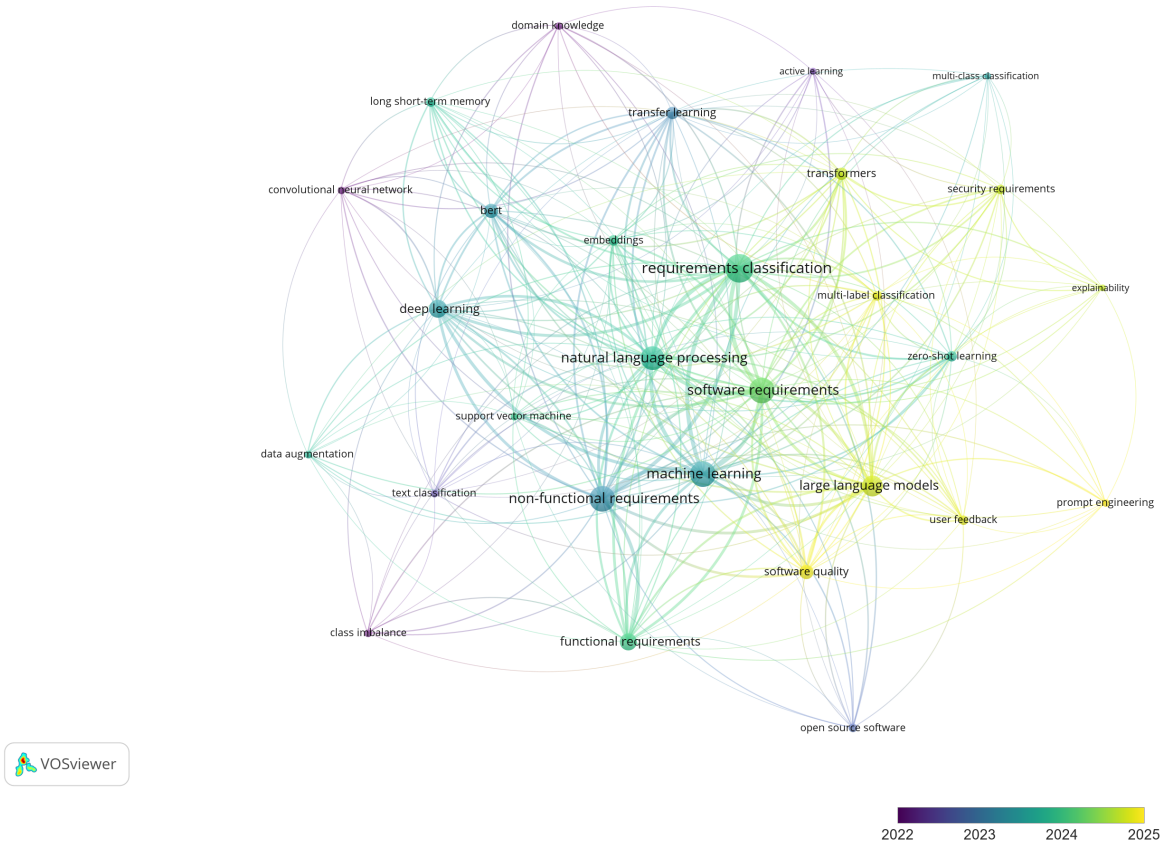


Fig. 3. Mapa de la Fig. 2 coloreado según el año promedio de publicación de los estudios que usan cada palabra clave, elaborado con VOSviewer [4]. Los valores fuera de 2022–2025 se muestran en el extremo de la escala.

Los términos con menor año promedio son *class imbalance* (2021.7), CNN (2022.8) y *domain knowledge* (2023.0). Los más recientes son *prompt engineering* (2025.2), *software quality* (2025.0) y *large language models*, *transformers*, *explainability* y *security requirements* (2024.8). Es decir, el desbalance se estudió sobre todo con las técnicas anteriores, y la línea más reciente, centrada en los LLM, apenas lo retoma.

III-B. Evolución temporal y enfoques

La producción es reciente y crece rápido: 90 de los 145 estudios (62 %) son de 2025 o 2026, aunque los datos de 2026 son parciales (Fig. 4). La figura también muestra el relevo de enfoques. El aprendizaje automático clásico está presente en todo el periodo, pero en menor proporción cada año. Los transformers aparecen en 2020 con NoRBERT [6] y son el enfoque principal de 68 estudios. Los LLM entran en 2022 y suman 45 estudios, 37 de ellos de 2025–2026. El aprendizaje profundo sin transformers queda en 10 estudios.

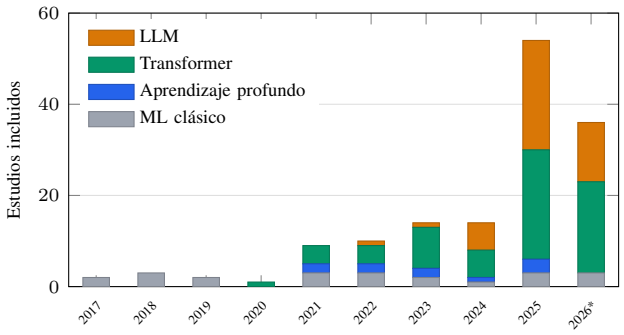


Fig. 4. Estudios incluidos por año y enfoque principal ($n = 145$). *Datos parciales de 2026.

III-C. Fuentes de texto y conjuntos de datos

Predomina el texto de especificaciones: 107 estudios trabajan con SRS o conjuntos derivados de PROMISE (Fig. 5). Las reseñas y el feedback de usuarios aparecen en 19 estudios, los issues de GitHub y otros proyectos abiertos en 16, y las historias de usuario en 3. Entre los conjuntos nombrados en los resúmenes destacan PROMISE (38), su extensión PROMISE_exp (12), PURE (8) y SecReq (5). Además, 48 estudios abordan la tarea binaria FR/NFR y 17 la clasificación multietiqueta. Esta variedad de conjuntos, particiones y tareas impide comparar directamente las cifras que reporta cada estudio.

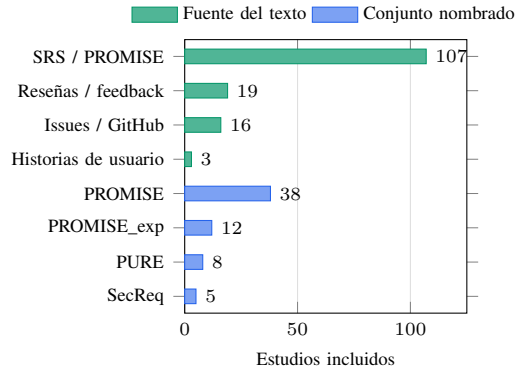


Fig. 5. Fuente del texto clasificado (una por estudio) y conjuntos de datos nombrados en los resúmenes.

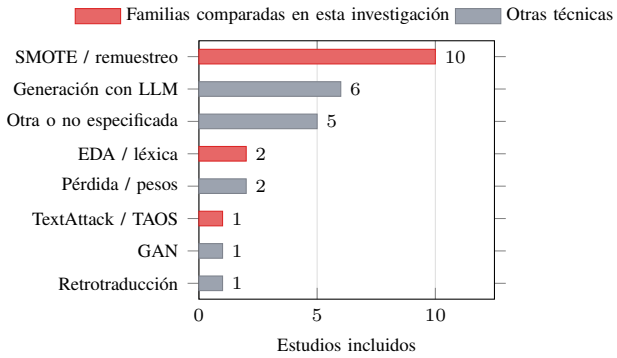


Fig. 6. Técnicas contra el desbalance en los estudios incluidos (un estudio puede usar más de una).

III-D. Desbalance y aumentación

Solo 46 de los 145 estudios (32 %) mencionan el desbalance, la escasez de datos o alguna técnica para tratarlos (Fig. 6). Entre las técnicas identificables predominan el remuestreo tipo SMOTE (10 estudios) y la generación de muestras con LLM (6). Les siguen la aumentación léxica tipo EDA (2), los ajustes en la función de pérdida (2) y la aumentación con GAN, con TextAttack y por retrotraducción (1 cada una). Otros 5 estudios usan aumentaciones propias o no especifican el método. Las familias de los tres métodos que compara esta investigación (remuestreo, aumentación léxica y TextAttack) suman 13 apariciones, pero ningún estudio combina dos de ellas.

III-E. Trabajos más co-citados

La Fig. 7 cuenta cuántos estudios incluidos citan cada trabajo base. Los más citados son de clasificación: Kurtanović y Maalej (71), NoRBERT (65), BERT [9] (55) y Cleland-Huang et al. (48). Los métodos de aumentación casi no se citan: SMOTE [10] aparece en 6 estudios, EDA [11] en 5 y TextAttack [12] en 1. La aumentación sigue siendo una línea secundaria dentro de la clasificación de requerimientos.

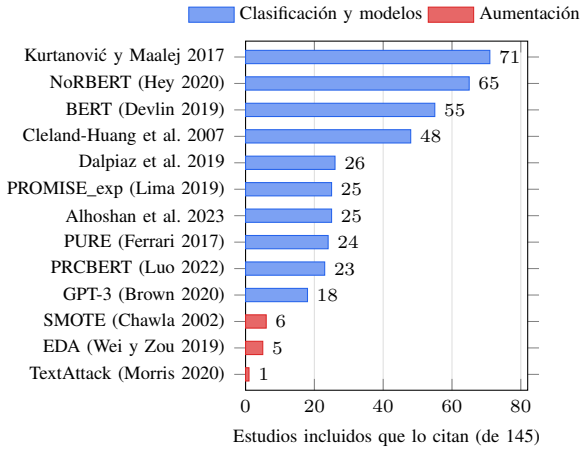


Fig. 7. Trabajos base más citados por los estudios incluidos.

IV. Evolución de la clasificación de NFR

IV-A. Aprendizaje automático clásico

Los primeros estudios representaron el texto con bolsas de palabras o TF-IDF y lo clasificaron con SVM, Naive Bayes o k-NN. Kurtanović y Maalej [5] clasificaron FR y NFR con SVM y usaron sub- y sobremuestreo para compensar el desbalance. Dalpiaz et al. [13] propusieron un clasificador interpretable basado en análisis de dependencias. Tóth y Vidács [14] compararon varios clasificadores considerando también su tiempo de ejecución. Younas et al. [15] identificaron NFR sin datos etiquetados, por similitud semántica con Word2Vec sobre PROMISE NFR. Binkhonain y Zhao [16] revisan esta etapa.

IV-B. Aprendizaje profundo

La segunda etapa dejó de lado la ingeniería manual de características. Rahimi et al. [17] combinaron LSTM, BiLSTM, GRU y CNN en sistemas de una y dos fases, con 95.7 % de exactitud en la tarea binaria y 93.4 % en la multiclase. Sabir et al. [18] entrenaron ANN, CNN, LSTM y GRU sobre cinco clases de NFR. Con una aumentación propia, la exactitud de la CNN pasó de 60 % a 96 % (Sección V). NFRNet [19], [20] combina un BERT con enmascaramiento de n-gramas y una BiLSTM, y amplía PROMISE con un nuevo conjunto, SOFTWARE NFR.

IV-C. Transformers y BERT

NoRBERT [6] trasladó el ajuste fino de BERT [9] a esta tarea y es el segundo trabajo más citado del corpus. Después se le añadieron capas y estructuras:

- DBGAT [21] combina BERT con una red de atención sobre grafos de dependencias y reporta un F1 de hasta 91 % en proyectos vistos y 88 % en no vistos.
- BERT-BiCNN [22] añade BiLSTM y CNN sobre BERT.
- PRCBERT [23] reformula la clasificación con plantillas de *prompt*.
- Kiani et al. [24] combinan BERT, DistilBERT y RoBERTa en un ensamble con fusión por atención (F1 de 0.94).

También aparecen modelos preentrenados con texto de requerimientos, como RequiBERT [25]. Este modelo reporta un F1 de 85 % en todas las subclases de NFR de PROMISE. Alhaizaey y Al-Mashari [8] trataron el desbalance en la función de pérdida (*Focal Loss*) y obtuvieron un F1 de 93.4 % con BERT sobre más de 10 000 historias de usuario.

Varios estudios recientes documentan los límites de BERT con las clases raras. Ayed et al. [26] evaluaron BERT en cinco conjuntos y encontraron errores sistemáticos en las clases con pocas muestras, como tolerancia a fallos y portabilidad. Concluyen que se necesitan estrategias de muestreo y aumentación. Nagaraju et al. [27] caracterizan el desbalance de PROMISE_exp (969 requerimientos: 444 FR y 525 NFR en doce categorías) y proponen una clasificación jerárquica en dos niveles. Kamal et al. [28] muestran que los modelos ajustados, como NoRBERT o PRCBERT, pierden desempeño al pasar de PROMISE a texto informal de Stack Overflow.

IV-D. LLM como clasificadores

Los LLM se usan sobre todo sin entrenamiento específico, en modo *zero-shot* o *few-shot*. Alhoshan et al. [29], [30] abrieron esta línea con aprendizaje *zero-shot*: en su estudio preliminar, la exhaustividad y el F1 promediaron 82 % al separar usabilidad de seguridad. El-Hajjami et al. [31] compararon GPT-3.5 y GPT-4 con SVM y LSTM en cinco conjuntos y concluyeron que ninguna técnica es la mejor para todas las clases. Zhang et al. [32] evaluaron seis LLM en las tareas binaria y multiclase. Varios trabajos estudian la sensibilidad de los LLM a su configuración:

- el número de ejemplos en el *prompt* [33];
- la cantidad de requerimientos por lote [34];
- la temperatura [35], [36];
- la estrategia de *prompting* [37], donde el método supervisado mantuvo la mejor exactitud global.

Los estudios que comparan LLM con transformers ajustados llegan a resultados que dependen del escenario:

- Binkhonain y Alfayez [7]: con *few-shot*, los LLM igualan o superan a un transformer ajustado en PROMISE y SecReq.
- Shafikuzzaman et al. [38]: NoRBERT sigue siendo superior en la tarea binaria FR/NFR, y los modelos *few-shot* lo igualan o superan en las demás.
- Wang et al. [39]: BERT fija el techo de exactitud con datos desbalanceados, mientras que los LLM destacan cuando importa la exhaustividad o hay pocos datos.

También hay ensambles de varios LLM [40] y evaluaciones de modelos abiertos [41], [42] (preprints). En ninguno de estos estudios el transformer de referencia se entrenó con datos aumentados.

V. Desbalance de clases y aumentación de datos

El desbalance degrada el desempeño en las clases minoritarias [55], y la aumentación de texto es una respuesta extendida en NLP [56], [57]. En los NFR, las técnicas se agrupan en cinco familias.

V-A. Remuestreo en el espacio de características: SMOTE y SMOTE-Tomek

SMOTE [10] sintetiza vectores de la clase minoritaria por interpolación entre vecinos, y los enlaces de Tomek limpian la frontera entre clases [58]. Es la familia más usada en el corpus, casi siempre en la tarea binaria FR/NFR:

- Hakim et al. [43] aplicaron SMOTE con RF, k-NN y SVM sobre 1,366 requerimientos (1,141 FR y 225 NFR).
- Vijayvargiya et al. [59] balancearon con SMOTE antes de ocho clasificadores profundos.
- Sonawane et al. [60] usaron SMOTE con una CNN optimizada.
- Or [45] (preprint) aplicó SMOTE-Tomek a 969 requerimientos de PROMISE: la regresión logística pasó de 59.85 % a 76.16 % de exactitud.
- Rubaidi et al. [61] abordaron el desbalance FR/NFR con algoritmos de ML.

Limaylla-Lunarejo et al. [53] combinaron sobre- y submuestreo en PROMISE_exp traducido al español: las clases raras mejoraron, aunque el desempeño global bajó. Como SMOTE opera sobre vectores, no produce requerimientos legibles. En los estudios revisados no se identificó su aplicación sobre representaciones de BERT para las 12 clases.

V-B. Aumentación léxica: EDA y variantes

EDA [11] genera variantes de una oración con cuatro operaciones: reemplazo por sinónimos, inserción, intercambio y eliminación aleatorios. En NFR aparece sobre todo como línea base. Sabir et al. [18], [44] compararon una aumentación propia, que concatena oraciones de la misma clase, con EDA y con un control sin aumentación. Al aumentar solo el conjunto de entrenamiento, la diferencia frente a EDA fue marginal. Ninguno de los dos estudios usó BERT.

V-C. Aumentación con TextAttack: TAOS

TextAttack [12] ofrece recetas de aumentación además de ataques adversarios. TAOS [3] lo usa con BERT base uncased, con 1,073 requerimientos de entrenamiento y 461 de prueba en 12 clases. Con TAOS, el F1 ponderado sube de 0.75 a 0.93 y el F1 macro de 0.44 a 0.89. Las clases MN, L, FT y PO, que el modelo base no reconocía, alcanzan valores de F1 de 0.50 a 1.00. Su lectura tiene dos límites:

- TAOS solo se compara con un BERT sin aumentación.
- Su Tabla III deja PO en 10 muestras frente a 782 de F (Fig. 8), por lo que el balanceo es parcial.

Para esta revisión se analizaron los archivos de entrenamiento y prueba que los autores de TAOS publicaron en su repositorio. Los conteos coinciden con su Tabla III: el entrenamiento contiene 2 requerimientos de PO y la prueba, 3. Sin embargo, numerosos requerimientos aparecen repetidos, una vez entre comillas simples y otra sin ellas, o con diferencias de espaciado. Al comparar los textos normalizados (minúsculas, sin puntuación ni espacios repetidos) se obtiene lo siguiente:

- 240 de los 461 requerimientos de prueba (52 %) también están en entrenamiento, 238 de ellos con la misma etiqueta.

Tabla II
ESTUDIOS SOBRE DESBALANCE Y AUMENTACIÓN DE DATOS EN LA CLASIFICACIÓN DE REQUERIMIENTOS

Estudio	**Año**	**Técnica contra el desbalance**	**Clasificador y datos**	**Resultado reportado**	**Limitación para esta investigación**
Hakim et al. [43]	2018	SMOTE	RF, k-NN, SVM; 1,366 req. FR/NFR	SVM, el mejor en la clase minoritaria	Binaria; sin BERT
Sabir et al. [18], [44]	2021–22	Aumentación propia frente a EDA	ANN, CNN, LSTM, GRU; 5 clases NFR	CNN: 60 % $\rightarrow$ 96 % de exactitud; diferencia marginal con EDA	Sin BERT; 5 clases
Or [45]	2025 (preprint)	SMOTE-Tomek	RL; PROMISE, 969 req. FR/NFR	Exactitud 59.85 % $\rightarrow$ 76.16 %	Binaria; sin BERT
Ben Ammar y Almatrafi [3]	2025	TAOS (TextAttack)	BERT base; 1,534 req., 12 clases	F1 ponderado 0.75 $\rightarrow$ 0.93; F1 macro 0.44 $\rightarrow$ 0.89	Solo frente a BERT sin aumentación; PO queda en 10
Suchi et al. [46]	2024	Generación con GPT-Neo	BiLSTM-SVM; PROMISE_exp $\rightarrow$ 6,783 req.	Exactitud 97 %	Binaria; sin BERT
Pham et al. [47]	2025	NFR sintéticos con GPT-4o	SVM, NB y BERT; 11 clases NFR	En PROMISE, BERT real 0.80 frente a 0.77 combinado y 0.51 sintético; el sintético ayuda al probar en PURE	F1 ponderado; sin comparar otros métodos
Qin y Peng [48]	2025	ChatNRC: LLM generador y discriminador	PROMISE-NFR, 4 categorías	F1 ponderado 0.96	Sin las clases raras
Wang et al. [49]	2025	Generación con LLM y pérdida de cola larga	TextCNN; EMSE multietiqueta	Supera a ML, DL y LLM	Multietiqueta; sin BERT
Naseer et al. [50]	2026	Retrotraducción (inglés–alemán)	BERT	Exactitud 94.3 %; F1 0.90	Un solo método
Kaur y Kaur [51]	2026	Aumentación contextual con BERT	Transformers y ensamble; 3,072 req.	Exactitud +0.2 a +3.76 puntos	Taxonomía propia; un método
Aldhafer et al. [52]	2026	Aumentación adaptativa (diccionarios y transformers)	BiGRU; PROMISE (10 clases)	F1 macro +6.7 %	Clasificador BiGRU
Limaylla-Lunarejo et al. [53]	2026	Sobre- y submuestreo	Seis algoritmos de ML; PROMISE_exp en español	F1 82 % (RL) sin balancear; con balanceo baja el global y mejoran PO y FT	Sin BERT

Tabla III
ESTUDIOS SOBRE CLASIFICADORES BERT Y LLM Y SOBRE CONJUNTOS AMPLIADOS

Estudio	**Año**	**Enfoque**	**Datos y tarea**	**Resultado reportado**	**Limitación para esta investigación**
Alhaizaey y Al-Mashari [8]	2025	BERT, RoBERTa, XLNet, DistilBERT con *Focal Loss*	$>$10 000 historias de usuario	BERT: F1 93.4 %	Desbalance tratado en la pérdida
Ayed et al. [26]	2025	BERT ajustado	5 conjuntos, multiclase	Errores sistemáticos en FT y PO	Sin aumentación; la recomienda
Binkhonain y Alfayez [7]	2025	LLM *zero/few-shot*, persona y CoT	PROMISE y SecReq	*Few-shot* iguala o supera al transformer ajustado	Transformer sin aumentación
Shafikuzzaman et al. [38]	2025	6 PLM y LLM *zero/few-shot*	PROMISE y SecReq; 5 tareas	NoRBERT, el mejor en la binaria; *few-shot* lo iguala en otras	Sin aumentación
Kamal et al. [28]	2025	Transformers ajustados frente a LLM	PROMISE y NFR-SO, entre plataformas	GPT-4o *few-shot* generaliza mejor	Sin aumentación
Wang et al. [39]	2026	ML, DL y LLM (con LoRA)	EMSE y ARBD multietiqueta	BERT, techo con desbalance; LLM con pocos datos	Sin aumentación
Kamal et al. [54]	2026	GitReq: minería de GitHub; 4 LLM *zero-shot*	6,302 req., 8 categorías	GPT-5.2: F1 macro 0.641	Taxonomía distinta de las 12 clases

- Las 1,534 filas corresponden a solo 988 requerimientos distintos.
- Sin duplicados, PO tiene 4 requerimientos y la razón F:PO es de 143:1.

Esa superposición entre entrenamiento y prueba puede inflar tanto la línea base como el resultado de TAOS, y el artículo

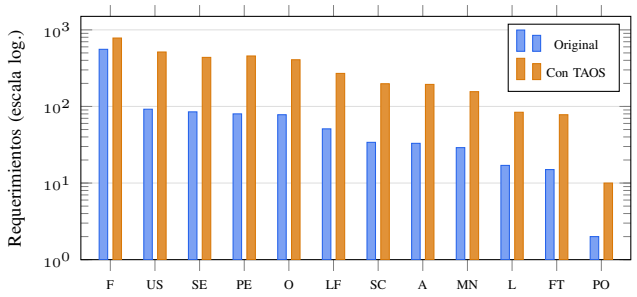


Fig. 8. Distribución de clases de entrenamiento antes y después de TAOS (Tabla III de [3]).

no la menciona. Además, con tan pocas muestras de prueba, un solo acierto o error cambia mucho el F1 de PO.

V-D. Retrotraducción y aumentación contextual

Naseer et al. [50] balancearon las clases minoritarias traduciendo al alemán y de vuelta al inglés, y luego clasificaron con BERT (F1 de 0.90). Kaur y Kaur [51] aplicaron aumentación contextual basada en BERT sobre 3,072 requerimientos de una taxonomía propia de dos niveles. La exactitud mejoró entre 0.2 y 3.76 puntos según el modelo. Aldhafer et al. [52] ajustan la cantidad de muestras sintéticas según la distribución de clases y la longitud de los requerimientos. Generan esas muestras con diccionarios de dominio y con modelos transformer, y clasifican con una BiGRU. Reportan un F1 macro 6.7 % mayor en la tarea de 10 clases de PROMISE. En los tres casos se evalúa un solo método de aumentación.

V-E. Generación con LLM

La generación con LLM es la línea más reciente y creciente:

- Suchi et al. [46] ampliaron PROMISE_exp con GPT-Neo hasta 6,783 requerimientos FR/NFR balanceados.
- Pham et al. [47] generaron NFR sintéticos por clase con GPT-4o. El texto completo matiza el resumen: al probar en PROMISE, BERT entrenado solo con datos reales (F1 ponderado 0.80) supera al combinado (0.77) y al sintético (0.51); la ventaja del sintético aparece al probar en otro conjunto, PURE.
- ChatNRC [48] usa un LLM como generador y discriminador sobre cuatro categorías de PROMISE-NFR.
- NFRLT [49] combina muestras generadas por LLM con una pérdida ajustada a la distribución de cola larga.
- KANFR-LT [62] genera muestras de las clases de cola guiado por un grafo de conocimiento.

Synthline [63] y O3DG [64] sistematizan la generación de conjuntos sintéticos de requerimientos. Fuera del dominio, AugGPT [65] y Møller et al. [66] documentan el mismo enfoque. En esta familia el LLM actúa como generador. Esa línea queda fuera del alcance de esta investigación, en la que los LLM solo clasifican, y la aumentación se limita a SMOTE-Tomek, EDA y TAOS.

VI. Ampliación de conjuntos de datos

Los conjuntos clásicos son pequeños, por lo que varios estudios buscan otras fuentes de texto. En reseñas de aplicaciones, MNoR-BERT [67] clasifica 6,000 reseñas en cuatro NFR con un enfoque multietiqueta. En proyectos abiertos, Pérez-Verdejo et al. [68] clasifican requerimientos de calidad a partir de issues de GitHub. Krasniqi y Cleland-Huang [69] detectan preocupaciones de calidad en resúmenes de reportes de defectos de proyectos de código abierto. GitReq [54] es el conjunto más cercano al objeto de esta investigación: reúne 6,302 requerimientos validados por expertos, minados de 55,588 candidatos en 4,080 repositorios. Usa ocho categorías alineadas con ISO/IEC 25010, con un acuerdo entre anotadores de $\kappa = 0{,}72$. Como línea base, el mejor LLM *zero-shot* (GPT-5.2) alcanzó un F1 macro de 0.641.

Otros estudios reducen el costo de etiquetar los datos nuevos:

- Aprendizaje semi-supervisado: Casamayor et al. [70] y Deocadez et al. [71], con auto-etiquetado de reseñas.
- GAN-BERT semi-supervisado [72].
- Aprendizaje activo [73], [74], incluso en un esquema federado [75].
- Auto-aprendizaje con PRCBERT [23]: transferido a datos sin etiquetas, logró un F1 *zero-shot* de 53.27 % y 72.96 % en dos conjuntos.

Estos métodos constituyen la base para ampliar un conjunto mediante pseudo-etiquetas. Aun así, las etiquetas asignadas por el propio modelo pueden propagar sus errores, y la taxonomía de GitReq no coincide con las 12 clases de PROMISE.

Tabla IV
Brechas de conocimiento y evidencia

ID	Brecha y evidencia
G1	**Sin comparación controlada de métodos de aumentación.** De los 46 estudios que tratan el desbalance, cada uno evalúa un solo método o lo compara con EDA sin usar BERT [44]. TAOS solo se contrasta con un BERT sin aumentación, y en una partición donde el 52 % de los requerimientos de prueba también está en entrenamiento [3]. No se identificó una comparación de TAOS, EDA y SMOTE-Tomek con el mismo BERT, sin duplicados y con los mismos pliegues.
G2	**Clases minoritarias sin resolver.** BERT falla de forma sistemática en tolerancia a fallos y portabilidad [26], y el balanceo de TAOS es parcial (PO pasa de 2 a 10 muestras) [3]. Sin duplicados, el conjunto tiene solo 4 requerimientos de PO. Además, 48 estudios se limitan a la tarea binaria FR/NFR.
G3	**LLM frente a BERT aumentado.** Los LLM se comparan con transformers ajustados sin aumentación, con ML clásico o con otros LLM [7], [28], [38], [39]. No se identificó una comparación con un BERT entrenado con datos aumentados en las 12 clases.
G4	**Heterogeneidad experimental.** Los estudios usan PROMISE (38), PROMISE_exp (12), PURE (8), SecReq (5) y conjuntos propios, con tareas binarias, multiclase o multietiqueta y métricas distintas, lo que impide comparar sus resultados.
G5	**Ampliación iterativa.** Existen un conjunto minado de GitHub con otra taxonomía [54] y métodos semi-supervisados y de aprendizaje activo [71], [74], [75], pero no se identificó un ciclo que amplíe el conjunto de 12 clases y repita la evaluación.

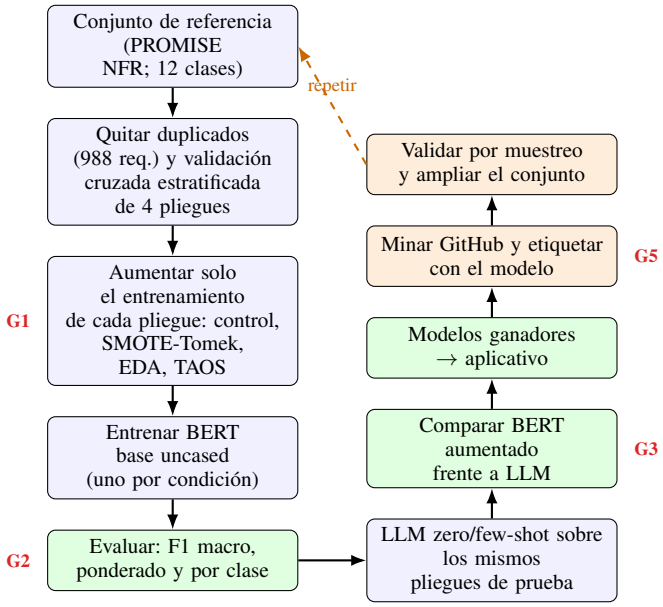


Fig. 9. Flujo experimental propuesto. En rojo, la brecha que aborda cada etapa.

VII. Análisis comparativo

Las Tablas II y III resumen los estudios más cercanos a esta investigación. En aumentación, cada estudio evalúa un único método, normalmente frente a un control sin aumentación. Además, la mayoría trabaja en la tarea binaria o con clasificadores distintos de BERT. En clasificación, los LLM se comparan con transformers ajustados o con ML clásico, nunca con un transformer entrenado con datos aumentados. Las cifras de ambas tablas no son comparables entre sí, porque cambian el conjunto, el número de clases, la partición y la métrica.

VIII. Discusión

PR1. Evolución y enfoques. Los transformers reemplazaron al ML clásico como enfoque principal, y los LLM crecen desde 2024. Aun así, SVM y regresión logística siguen apareciendo como líneas base. Varias de las cifras más altas, con F1 o exactitud por encima de 0.9, vienen de la tarea binaria FR/NFR [25], [46], [60]. Cuando el modelo se prueba en otra plataforma, el desempeño cae [28]. Por eso esas cifras dicen poco sobre el problema de 12 clases con clases raras.

PR2. Desbalance y aumentación. Solo un tercio de los estudios (46) trata el desbalance o la escasez de datos. Cuando lo hacen, cada estudio evalúa una técnica aislada, frente a un control sin aumentación o frente a EDA. EDA es una referencia habitual en las revisiones de aumentación de texto [56], [57], pero casi no se usa en NFR, y SMOTE se aplica sobre todo a vectores clásicos en la tarea binaria. También importa el protocolo. Sabir et al. [44] observaron que aumentar también el conjunto de validación elevaba los resultados de la CNN en 10 %. Esto sugiere que, si la aumentación se aplica antes de dividir los datos, las métricas pueden inflarse. Una comparación justa debe aumentar solo la parte de entrenamiento de cada pliegue.

PR3. LLM como clasificadores. Los LLM sin entrenamiento específico compiten con los modelos ajustados cuando hay pocos datos o cuando el texto viene de otra fuente [28], [39]. Con datos suficientes, el ajuste supervisado mantiene la ventaja, sobre todo en la tarea binaria [37], [38]. Además, los resultados de los LLM cambian con el número de ejemplos, la temperatura y el tamaño del lote [33], [34], [36]. Por ello, estos parámetros deben fijarse y reportarse. La pregunta que queda abierta es si un LLM sigue siendo competitivo frente a un BERT que recibió el mejor tratamiento de datos disponible, y no solo frente a uno entrenado con los datos originales.

PR4. Datos y ampliación. PROMISE y sus variantes concentran la evaluación, lo que favorece la comparación pero limita la variedad y el tamaño de las clases raras. GitReq [54] aporta volumen y validación experta, pero usa otra taxonomía, de modo que su incorporación a las 12 clases exige un mapeo explícito. El aprendizaje semi-supervisado y el aprendizaje activo reducen el costo de etiquetar, pero las pseudo-etiquetas pueden repetir los errores del modelo. Por eso cada ampliación necesita validación humana por muestreo.

IX. Brechas identificadas

Las brechas se formulan respecto a los 145 estudios revisados y se resumen en la Tabla IV. Las afirmaciones del tipo "no se identificó" se refieren a esos estudios.

X. Implicaciones para la investigación propuesta

La Fig. 9 presenta el flujo experimental que responde a las brechas:

- **Comparación controlada (G1, G4).** Primero se eliminan los requerimientos duplicados del conjunto de referencia (988 distintos). Luego se fijan el clasificador (BERT base uncased) y una validación cruzada estratificada de 4 pliegues, con los mismos pliegues para todas las condiciones. Se usan 4 pliegues porque PO tiene 4 requerimientos: así, cada pliegue de prueba contiene uno. En cada pliegue solo se aumenta la parte de entrenamiento, para evitar fuga de información. Se comparan cuatro condiciones: un control sin aumentación, SMOTE-Tomek, EDA y TAOS.
- **Métricas por clase (G2).** La evaluación usa F1 macro, ponderado y por clase, con atención a MN, L, FT y PO. Como cada pliegue de prueba tiene un solo requerimiento de PO, su F1 se calcula sobre las predicciones de todos los pliegues juntos y se reporta con la variación entre pliegues. La validación se repite con varias semillas.
- **LLM como clasificadores (G3).** Uno o varios LLM clasifican los mismos pliegues de prueba en *zero-shot* y *few-shot*. Los ejemplos del *prompt* salen solo de la parte de entrenamiento de cada pliegue. Los LLM actúan únicamente como clasificadores.
- **Ampliación (G5).** Los modelos ganadores alimentan un aplicativo de predicción y el ciclo de ampliación con GitHub, con validación humana por muestreo.

XI. Conclusiones

En los 145 estudios revisados, la clasificación de NFR pasó del ML clásico a los transformers y, desde 2024, a los LLM. La mayor parte de la producción es de 2025–2026 (PR1). El desbalance se reconoce como un problema central, pero solo un tercio de los estudios lo trata. Cuando lo hacen, cada uno evalúa un único método: SMOTE en la tarea binaria, generación con LLM o aumentaciones propias (PR2). Los LLM se usan sobre todo como clasificadores *zero-shot* y *few-shot*, y su ventaja depende del escenario. Sin embargo, se comparan con transformers que no fueron entrenados con datos aumentados (PR3). La mayoría de los estudios usa PROMISE y sus variantes; GitReq y el aprendizaje semi-supervisado y activo abren el camino para ampliar los conjuntos (PR4). La comparación controlada de TAOS, EDA y SMOTE-Tomek con BERT, contrastada con LLM, ocupa un espacio que los estudios revisados dejan abierto. Esa comparación debe hacerse sobre el conjunto sin duplicados, porque la partición pública de TAOS mezcla entrenamiento y prueba.

Limitaciones. Los estudios proceden de Scopus, por lo que pueden faltar trabajos indexados solo en otras bases. Los resultados atribuidos a cada estudio, salvo los de TAOS, provienen de sus resúmenes, y las categorías del panorama se asignaron por palabras clave con verificación manual. La revisión sistemática asociada a esta investigación extrae los datos del texto completo de los estudios disponibles y evalúa su calidad.

Disponibilidad de datos. La lista de estudios revisados, su caracterización y el código del análisis bibliométrico y de la verificación de TAOS están disponibles en el repositorio del proyecto (https://github.com/AlessandroPastor/research).

Referencias

[1] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, "Automated classification of non-functional requirements," *Requirements Engineering*, vol. 12, no. 2, pp. 103–120, 2007.

[2] J. Cleland-Huang, S. Mazrouee, H. Liguo, and D. Port, "nfr," Zenodo dataset, 2007.

[3] B. Ben Ammar and N. F. Almatrafi, "Classification of requirements from software requirement specification documents: TAOS: A novel TextAttack oversampling approach for fine-grained classification of software requirements," *Engineering, Technology & Applied Science Research*, vol. 15, no. 6, pp. 29 872–29 879, 2025.

[4] N. J. van Eck and L. Waltman, "Software survey: VOSviewer, a computer program for bibliometric mapping," *Scientometrics*, vol. 84, no. 2, pp. 523–538, 2010.

[5] Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *2017 IEEE 25th International Requirements Engineering Conference (RE)*, 2017, pp. 490–495.

[6] T. Hey, J. Keim, A. Koziolek *et al.*, "NoRBERT: Transfer learning for requirements classification," in *2020 IEEE 28th International Requirements Engineering Conference (RE)*, 2020, pp. 169–179.

[7] M. Binkhonain and R. Alfayez, "Are prompts all you need? Evaluating prompt-based large language models (LLM)s for software requirements classification," *Requirements Engineering*, vol. 30, no. 4, pp. 423–443, 2025.

[8] A. Alhaizaey and M. Al-Mashari, "Automated classification and identification of non-functional requirements in agile-based requirements using pre-trained language models," *IEEE Access*, vol. 13, pp. 87 401–87 417, 2025.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186.

[10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[11] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 6382–6388.

[12] J. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 119–126.

[13] F. Dalpiaz, D. Dell'Anna, F. B. Aydemir *et al.*, "Requirements classification with interpretable machine learning and dependency parsing," in *2019 IEEE 27th International Requirements Engineering Conference (RE)*, 2019, pp. 142–152.

[14] L. Tóth and L. Vidács, "Study of various classifiers for identification and classification of non-functional requirements," in *Lecture Notes in Computer Science*, 2018, pp. 492–503.

[15] M. Younas, K. Wakil, D. N., M. Arif, and A. Mustafa, "An automated approach for identification of non-functional requirements using word2vec model," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 8, 2019.

[16] M. Binkhonain and L. Zhao, "A review of machine learning algorithms for identification and classification of non-functional requirements," *Expert Systems with Applications: X*, vol. 1, p. 100001, 2019.

[17] N. Rahimi, F. Eassa, and L. Elrefaei, "One- and two-phase software requirement classification using ensemble deep learning," *Entropy*, vol. 23, no. 10, p. 1264, 2021.

[18] M. Sabir, E. Banissi, and M. Child, "A deep learning-based framework for the classification of non-functional requirements," in *Advances in Intelligent Systems and Computing*, 2021, pp. 591–601.

[19] B. Li, Z. Li, and Y. Yang, "Nfrnet: A deep neural network for automatic classification of non-functional requirements," in *2021 IEEE 29th International Requirements Engineering Conference (RE)*, 2021, pp. 434–435.

[20] B. Li and X. Nong, "Automatically classifying non-functional requirements using deep neural network," *Pattern Recognition*, vol. 132, p. 108948, 2022.

[21] G. Li, C. Zheng, M. Li, and H. Wang, "Automatic requirements classification based on graph attention network," *IEEE Access*, vol. 10, pp. 30 080–30 090, 2022.

[22] K. Kaur and P. Kaur, "Improving bert model for requirements classification by bidirectional lstm-cnn deep model," *Computers and Electrical Engineering*, vol. 108, p. 108699, 2023.

[23] X. Luo, Y. Xue, Z. Xing, and J. Sun, "Prcbert: Prompt learning for requirement classification using bert-based pretrained language models," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[24] A. A. Kiani, "A deep learning approach to classifying software requirements: The application of transformer-based ensemble learning and attention-based fusion," *Foundations of Computing and Decision Sciences*, vol. 51, no. 1, pp. 71–110, 2026.

[25] K. Rahman, A. Ghani, A. U. Rahman, S. Misra, and D. H. Kim, "Requibert: Transfer learning for software requirements classifications," *Expert Systems with Applications*, vol. 327, p. 132726, 2026.

[26] N. E. B. Ayed and M. Alrammal, "Benchmarking bert for software requirements classification across diverse datasets," in *2025 IEEE/ACS 22nd International Conference on Computer Systems and Applications (AICCSA)*, 2025, pp. 1–6.

[27] R. Nagaraju and V. Anantha Natarajan, "Hierarchical software requirement classification using transformer-based language models," *International Journal of Artificial Intelligence and Machine Learning*, vol. 6, no. 6s, pp. 340–359, 2026.

[28] F. Kamal and M. R. Islam, "Robust or overfitted? investigating the generalization of pretrained models in requirement classification," in *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2025, pp. 414–420.

[29] W. Alhoshan, L. Zhao, A. Ferrari, and K. J. Letsholo, "A zero-shot learning approach to classifying requirements: A preliminary study," in *Lecture Notes in Computer Science*, 2022, pp. 52–59.

[30] W. Alhoshan, A. Ferrari, and L. Zhao, "Zero-shot learning for requirements classification: An exploratory study," *Information and Software Technology*, vol. 159, p. 107202, 2023.

[31] A. El-Hajjami, N. Fafin, and C. Salinesi, "Which ai technique is better to classify requirements? an experiment with svm, lstm, and chatgpt," in *CEUR Workshop Proceedings*, vol. 3672, 2024.

[32] Y. Zhang and Y. Li, "The classification of software requirements using large language models: Emerging results and future directions," in *2024 3rd International Conference on Cloud Computing, Big Data Application and Software Engineering (CBASE)*, 2024, pp. 746–749.

[33] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, "The few-shot dilemma: Over-prompting large language models," in *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, 2025, pp. 134–141.

[34] A. T. Van Can, F. B. Aydemir, and F. Dalpiaz, "One size does not fit all: On the role of batch size in classifying requirements with llms," in *2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW)*, 2025, pp. 30–39.

[35] F. Karlsson, P. Chatzipetrou, S. Gao, and T. E. Havstorm, "Exploring classification consistency of natural language requirements using gpt-4o," in *Lecture Notes in Business Information Processing*, 2025, pp. 44–50.

[36] ——, "How reliable are gpt-4o and llama3.3-70b in classifying natural language requirements?: The impact of the temperature setting," *IEEE Software*, vol. 42, no. 6, pp. 97–104, 2025.

[37] L. Zhu, "Prompt engineering strategies for automated software requirements classification," in *2026 7th International Conference on Big Data & Artificial Intelligence & Software Engineering (ICBASE)*, 2026, pp. 546–552.

[38] M. Shafikuzzaman, M. R. Islam, S. Zaman, A. Ma, and A. Islam Sifat, "On the effectiveness of zero-shot and few-shot pretrained language models for software requirement classification," *IEEE Access*, vol. 13, pp. 159 439–159 453, 2025.

[39] W. Wang, J. Peng, H. Xiao, Y. Hua, Y. Zhou, Z. Li, and X. Wang, "Machine learning, deep learning, or large language models: An empirical study on multi-label requirements classification," *Empirical Software Engineering*, vol. 31, no. 6, p. 149, 2026.

[40] M. Kassab and W. Belbach, "Automating requirements classification: A systematic literature review and ensemble LLM-based approach," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering (EASE) Companion*, 2025, pp. 82–90.

[41] W. Alhoshan, A. Ferrari, and L. Zhao, "How effective are generative large language models in performing requirements classification?" arXiv:2504.16768, 2025.

[42] E. E. Sakib, M. A. Akib, M. M. Mazumder *et al.*, "A benchmark dataset and LLMs comparison for NFR classification with explainable AI," arXiv:2510.18096, 2025.

[43] L. Hakim and S. Rochimah, "Oversampling imbalance data: Case study on functional and non functional requirement," in *2018 Electrical Power, Electronics, Communications, Controls and Informatics Seminar (EECCIS)*, 2018, pp. 315–319.

[44] M. Sabir, E. Banissi, and M. Child, "Custom data augmentation technique (a deeper insight)," in *Lecture Notes in Networks and Systems*, 2022, pp. 75–80.

[45] B. Or, "Improving requirements classification with SMOTE-Tomek preprocessing," arXiv:2501.06491, 2025.

[46] K. N. Jahan Suchi, N. Jahan, A. Tasnim, and N. Akhter, "Explainable attention based bilstm-svm for software requirement classification: Integrating generative artificial intelligence," in *2024 27th International Conference on Computer and Information Technology (ICCIT)*, 2024, pp. 2404–2409.

[47] T. K. B. Pham, A. Alshomar, S. Supakkul, T. Hill, and L. Chung, "Generating synthetic nonfunctional requirements using large language models for training machine learning classifiers (s)," in *International Conferences on Software Engineering and Knowledge Engineering*, vol. 2025, 2025, pp. 217–222.

[48] Y. Qin and R. Peng, "ChatNRC: A non-functional requirement classification framework based on a generative and discriminative mechanism," in *2025 32nd Asia-Pacific Software Engineering Conference (APSEC)*, 2025, pp. 467–478.

[49] W. Wang, Z. Huang, H. Xiao, Z. Li, and B. Huang, "Learning in imperfect environments: Multi-label classification of non-functional requirements under long-tailed distribution," in *2025 25th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*, 2025, pp. 609–618.

[50] M. Naseer, U. Qamar, and S. Almutairi, "A fused back translation and bert model for optimizing software requirements classification," *Arabian Journal for Science and Engineering*, 2026.

[51] J. Kaur and C. Roy, "Hybrid approach for multilevel multi-class requirement classification: Impact of stop-word removal and data augmentation," *Journal of Systems and Software*, vol. 231, p. 112594, 2026.

[52] O. Q. Aldhafer, H. A. Alsadiq, M. Alshaboti *et al.*, "Adaptive data augmentation techniques for software requirements classification using deep learning," *IEEE Access*, vol. 14, pp. 43 841–43 854, 2026.

[53] M. Limaylla-Lunarejo, N. Condori-Fernandez, M. Rodríguez Luaces, and O. Karras, "Improving the multi-class classification of non-functional requirements in Spanish: A study of dataset balancing and performance," *Empirical Software Engineering*, vol. 31, 2026.

[54] F. Kamal, M. H. Kabir, and M. R. Islam, "Gitreq: A gold standard dataset for software quality requirements," in *2026 IEEE/ACIS 24th International Conference on Software Engineering Research, Management and Applications (SERA)*, 2026, pp. 116–121.

[55] J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *Journal of Big Data*, vol. 6, 2019.

[56] S. Y. Feng, V. Gangal, J. Wei *et al.*, "A survey of data augmentation approaches for NLP," in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021, pp. 968–988.

[57] M. Bayer, M.-A. Kaufhold, and C. Reuter, "A survey on data augmentation for text classification," *ACM Computing Surveys*, vol. 55, no. 7, 2022.

[58] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 20–29, 2004.

[59] S. Vijayvargiya, L. Kumar, L. B. Murthy, and S. Misra, "Software requirements classification using deep-learning approach with various hidden layers," in *Annals of Computer Science and Information Systems*, vol. 30, 2022, pp. 895–904.

[60] S. N. Sonawane and S. M. Puthran, "Classification of functional and nonfunctional requirements based on convolutional neural network with flower pollination optimizer," *Innovations in Systems and Software Engineering*, vol. 21, no. 3, pp. 1041–1065, 2025.

[61] Z. S. Rubaidi, B. Ben Ammar, and M. Ben Aouicha, "Handling imbalance functional and non-functional software requirement classification based on machine learning algorithms," in *Lecture Notes in Networks and Systems*. Springer Nature Switzerland, 2025, pp. 199–209.

[62] J. Peng, W. Wang, S. Tan, and Z. Li, "Knowledge-aware semantic complementation and semantic reasoning for long-tail nfr classification," in *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*, 2026, pp. 194–199.

[63] A. El-Hajjami and C. Salinesi, "Synthline: A product line approach for synthetic requirements engineering data generation using large language models," in *Lecture Notes in Business Information Processing*, 2025, pp. 208–225.

[64] G. Bishop, S. Chakraborty, H. Zhou, J. Dehlinger, L. Deng, J. Lin, and B. Kist, "On-demand generation of high-quality software engineering datasets using large language models and ontologies," *Automated Software Engineering*, vol. 33, no. 3, p. 73, 2026.

[65] H. Dai *et al.*, "AugGPT: Leveraging ChatGPT for text data augmentation," arXiv:2302.13007, 2023.

[66] A. G. Møller, J. A. Dalsgaard, A. Pera, and L. M. Aiello, "The parrot dilemma: Human-labeled vs. LLM-augmented data in classification tasks," arXiv:2304.13861, 2023.

[67] K. Kaur and P. Kaur, "Mnor-bert: multi-label classification of non-functional requirements using bert," *Neural Computing and Applications*, vol. 35, no. 30, pp. 22 487–22 509, 2023.

[68] J. M. Pérez-Verdejo, Á. J. Sánchez-García, and J. O. Ocharán-Hernández, "Requirements and GitHub issues: An automated approach for quality requirements classification," *Programming and Computer Software*, vol. 47, no. 8, pp. 704–721, 2021.

[69] R. Krasniqi and A. Agrawal, "Analyzing and detecting emerging quality-related concerns across oss defect report summaries," in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2021, pp. 12–23.

[70] A. Casamayor, D. Godoy, and M. Campo, "Identification of non-functional requirements in textual specifications: A semi-supervised learning approach," *Information and Software Technology*, vol. 52, no. 4, pp. 436–445, 2010.
[71] R. Deocadez, R. Harrison, and D. Rodriguez, "Automatically classifying requirements from app stores: A preliminary study," in *2017 IEEE 25th International Requirements Engineering Conference Workshops (REW)*, 2017, pp. 367–371.
[72] G. Airlangga, "Enhancing software requirements classification with semisupervised gan-bert technique," *Journal of Electrical and Computer Engineering*, vol. 2024, no. 1, p. 4955691, 2024.
[73] C. Magalhaes, J. Araujo, and A. Sardinha, "Mare: an active learning approach for requirements classification," in *2021 IEEE 29th International Requirements Engineering Conference (RE)*, 2021, pp. 516–521.
[74] Y. Zhang, W. Zou, C. Li, and B. Luo, "Reducing workload of manual annotation for user requests via a novel active learning framework," in *2022 International Conference on Industrial IoT, Big Data and Supply Chain (IIoTBDSC)*, 2022, pp. 214–225.
[75] W. Alhoshan, "Seqfal: A federated active learning framework for private and efficient labeling of security requirements," *Applied Sciences*, vol. 16, no. 2, p. 914, 2026.